HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO PROJECT 01

DIABETES AGENT

HỆ ĐA TÁC TỬ HỖ TRỢ SÀNG LỌC VÀ TRA CỨU KIẾN THỨC TIỂU ĐƯỜNG HỌC SÂU, TRI THỨC VÀ HỌC TĂNG CƯỜNG

Họ và tên:	Bùi Anh Đức
Lớp:	D23CTPM01
Mã sinh viên:	B23DCCN165
Học kỳ:	I năm học 2026 – 2027
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế

Hà Nội, tháng 11/2026

MỤC LỤC

1 Giới thiệu và phạm vi đề tài **1**

1.1 Bài toán đặt ra . . . 1

1.2 Mục tiêu và sản phẩm bàn giao . . . 1

1.3 Cách kế thừa tài liệu và workspace . . . 2

1.4 Phạm vi được hiện thực . . . 2

2 Cơ sở tác tử và kiến trúc hệ thống **3**

2.1 Tác tử và hệ đa tác tử . . . 3

2.2 Kiến trúc tổng thể . . . 3

2.3 Vai trò và hợp đồng đầu ra . . . 4

2.4 Thông điệp và điều kiện dừng . . . 5

2.5 Luật ưu tiên và quyền kiểm tra . . . 5

3 Dữ liệu và mô hình học sâu **6**

3.1 Nguồn dữ liệu và ý nghĩa nhãn . . . 6

3.2 Đặc trưng và mã hóa . . . 6

3.3 Chia tập và chống rò rỉ . . . 7

3.4 MLP PyTorch và hàm mất mát . . . 7

3.5 Các mô hình đối chứng . . . 8

3.6 Ngưỡng và chỉ số đánh giá . . . 8

4 Tri thức và học tăng cường **9**

4.1 Quy trình xây dựng kho tri thức . . . 9

4.2 Đồ thị khái niệm local . . . 9

4.3 Truy xuất và kết xuất câu trả lời . . . 10

4.4 Thiết kế đánh giá truy xuất . . . 11

4.5 MDP nhắc ghi nhật ký . . . 11

4.6 Huấn luyện Q-learning . . . 11

4.7 Kết hợp luật với chính sách . . . 12

5 Mã nguồn và notebook thực hành **13**

5.1 Cấu trúc thư mục . . . 13

5.2 Các hàm và lớp chính . . . 13

5.3 Notebook 01 từ dữ liệu tới model đã lưu . . . 14

5.4 Notebook 02 từ query tới nguồn . . . 14

5.5 Notebook 03 học và kiểm tra policy . . . 15

5.6 Notebook 04 tích hợp và kiểm tra veto . . . 16

5.7 Lưu trữ và chế độ local 17

6 Thực nghiệm và phân tích kết quả 18

6.1 Điều kiện thực nghiệm 18
6.2 Kết quả validation và lựa chọn mặc định 18
6.3 Kết quả trên test đã khóa 19
6.4 Quá trình học của MLP 20
6.5 Truy xuất tri thức và vai trò đồ thị 20
6.6 Kết quả Q-learning và đối chứng 21
6.7 Kiểm tra tích hợp và lỗi được inject 22
6.8 Kiểm thử tự động và notebook 22

7 Chạy local và trình diễn hệ thống 23

7.1 Cài đặt môi trường 23
7.2 Các tab và luồng thao tác 23
7.3 Ví dụ một lượt hội chẩn 24
7.4 Kịch bản bảo vệ ngắn 25
7.5 Tái lập thực nghiệm và notebook 25
7.6 Xử lý lỗi thường gặp 26
7.7 Biên dịch lại báo cáo 26

8 Kết luận và hướng phát triển 27

8.1 Kết quả đạt được 27
8.2 Giới hạn hiện tại 27
8.3 Hướng phát triển có thể kiểm chứng 27

Tài liệu tham khảo 28

DANH MỤC HÌNH

2.1 Các nguồn thông tin và cơ chế phối hợp của Diabetes Agent. 4

4.1 Đồ thị 12 khái niệm và các quan hệ được biên soạn trong project. Các ghi chú D01–D12 được gắn cạnh bằng chứng trong JSON. 10

5.1 Cell huấn luyện thật trong notebook 01 và một phần output đã lưu. Nội dung ảnh được dựng từ source/output của ipynb đã thực thi. 14

5.2 Ví dụ truy xuất trong notebook 02; ID nguồn gắn với đoạn được trả về. 15

5.3 Cell đọc Q-table thực tế của notebook 03 và chính sách cho tám trạng thái. . . . 16

5.4 Kiểm tra nhánh urgent trong notebook 04; trace cho thấy chưa gọi mô hình Risk. 17

6.1 ROC và Precision–Recall trên test. Mọi quyết định về model/ngưỡng được thực hiện bằng validation trước đó. 19

6.2 Loss train và validation, đường dọc thể hiện epoch được chọn để lưu checkpoint. 20

6.3 Đường học năm seed và reward đánh giá trong môi trường nhắc việc giả lập. . 21

7.1 Giao diện local sau khi chạy hội chẩn tổng hợp. Ảnh chụp từ trình duyệt của bản Project01. 24

7.2 Tab tri thức với query, ghi chú và URL nguồn để kiểm tra lại căn cứ. 25

DANH MỤC BẢNG

1.1 Tiêu chí hoàn thành và cách kiểm tra. 2

2.1 Liên hệ loại tác tử và hiện thực trong Project01. 3
2.2 Bảy vai trò tính cả điều phối. 4
2.3 Ba performative được hiện thực thực tế. 5

3.1 Nhóm đặc trưng và những điểm dễ nhập sai. 6
3.2 Phân bố tập nghiên cứu được sử dụng thực tế. 7
3.3 Cấu hình huấn luyện MLP được khóa trong source. 8

4.1 Danh mục 12 ghi chú có liên kết tới bảy trang nguồn. 9

5.1 Đường đi từ yêu cầu tới source code. 13

6.1 Chỉ số trên validation; ngưỡng được chọn bằng F1 validation. 18
6.2 Chỉ số trên test; ngưỡng được chọn bằng F1 validation. 19
6.3 Ngưỡng đã khóa và số lượng dự đoán trên test. 19
6.4 Truy xuất trên 24 câu do tác giả thiết kế; không phải kiểm định bên ngoài. . . . 20
6.5 Reward mỗi ngày trong simulator; std giữa năm giá trị trung bình theo seed train. 21
6.6 Các kịch bản chức năng được chạy ở phiên bản bàn giao. 22

7.1 Lỗi vận hành và cách xử lý tương ứng. 26

Phần 1

Giới thiệu và phạm vi đề tài

1.1 Bài toán đặt ra

Project01 xây dựng một hệ đa tác tử mang tên Diabetes Agent để minh họa cách kết hợp dữ liệu có cấu trúc, mô hình học sâu, tri thức có nguồn và một chính sách học tăng cường trong cùng ứng dụng local. Người sử dụng có thể chạy phân loại trên hồ sơ khảo sát, hỏi kiến thức về tiểu đường, xem hành động nhắc ghi nhật ký và theo dõi các bước điều phối. Kết quả quan trọng của project là một hệ thống chạy được, có mã nguồn và notebook tái lập; chất lượng được đánh giá riêng cho mô hình, truy xuất và hợp đồng phần mềm.

Một câu trả lời về sức khỏe có thể chứa nhiều loại thông tin với yêu cầu kiểm tra khác nhau. Điểm phân loại phải đến từ model đã huấn luyện; thông tin giáo dục cần liên kết tới nguồn; hành động nhắc việc phải nằm trong phạm vi cho phép. Nếu gộp tất cả vào một chuỗi xử lý không quan sát được, người đọc khó biết kết luận đến từ dữ liệu, từ luật hay từ nội dung biên soạn. Việc chia thành các tác tử giúp xác định trách nhiệm và kiểm tra đầu ra của từng bước.

Project sử dụng các nguồn giáo dục sức khỏe chính thức NIDDK và CDC [7, 10]. Các nguồn này được dùng để biên soạn ghi chú ngắn và đặt giới hạn sử dụng, không để tuyên bố hệ thống đã được thẩm định lâm sàng. Công cụ không chẩn đoán, kê đơn, điều chỉnh thuốc hoặc thay thế tư vấn của nhân viên y tế.

1.2 Mục tiêu và sản phẩm bàn giao

Sản phẩm cần giải quyết bốn nhóm yêu cầu. Thứ nhất, từ CSDL nghiên cứu có sẵn, huấn luyện MLP để phân loại nhãn nhị phân và đối chiếu với mô hình đơn giản hơn. Thứ hai, lưu trữ tri thức có ID và URL, truy xuất theo câu hỏi rồi kết xuất câu trả lời có bằng chứng. Thứ ba, hiện thực Q-learning cùng luật ràng buộc để minh họa một tác tử học chính sách. Thứ tư, phối hợp các kết quả qua điều phối, kiểm tra và tư vấn, đồng thời lưu được vết trao đổi khi cần.

Các sản phẩm cụ thể gồm mã Python chia theo chức năng, bốn notebook có output thực, dữ liệu và model đã huấn luyện, demo Streamlit chạy tại localhost, bộ kiểm thử và báo cáo này cùng source LaTeX. Người nhận có thể mở demo ngay sau khi cài thư viện; không cần API key và không cần huấn luyện lại trước buổi trình diễn.

Bảng 1.1: Tiêu chí hoàn thành và cách kiểm tra.

Yêu cầu	Bằng chứng kiểm tra
Source code rõ ràng	Các module trong `diabetes_agent/`; notebook và app gọi cùng mã.
Notebook thực thi được	Bốn file `ipynb`, execution count và output; log tại `notebook_execution.json`.
Mô hình học thật	Weight PyTorch, scaler, baseline, history và prediction test từng dòng.
Demo local	Bốn tab giao diện, chạy hội chẩn, tải trace JSON; kiểm tra bằng trình duyệt và AppTest.
Báo cáo phù hợp	Bìa PTIT và hệ thống định dạng theo Agent02; bảng/hình lấy từ artifact Project01.

1.3 Cách kế thừa tài liệu và workspace

Báo cáo *project01_CT_03_khanhtv.pdf* do người dùng cung cấp được tham khảo về cấu trúc bài toán: nhánh dữ liệu và học sâu, nhánh KB và truy xuất, nhánh RL kết hợp luật, sau đó hội tụ ở hệ đa tác tử. Project này không sử dụng các số liệu thực nghiệm của tài liệu đó làm kết quả của mình. Không triển khai giả định hiệu quả thuốc hoặc hiệu chỉnh dân số chỉ để giống báo cáo mẫu.

Tập nghiên cứu 24.000 dòng CDC được kế thừa nguyên dữ liệu và split từ dự án `Diabetes_MultiAgent` có sẵn trong workspace. Các model của Project01 được huấn luyện mới và đóng gói riêng. CSV, source, model và hướng dẫn đều nằm trong thư mục bàn giao, nên khi di chuyển thư mục không cần tiếp tục truy cập project cũ.

Hình thức báo cáo bám theo Agent01/Agent02: khung bìa và logo PTIT, Times New Roman cỡ 12, lề trái 3 cm, phải 2 cm, trên dưới 2,5 cm, header khoa và nhóm, số trang giữa footer, tiêu đề chia theo Phần, bảng và hình có chú thích. Thông tin sinh viên được giữ theo báo cáo hiện có: Bùi Anh Đức, B23DCCN165, D23CTPM01.

1.4 Phạm vi được hiện thực

Nhánh dự đoán sử dụng 21 chỉ báo khảo sát và nhãn gộp tiền tiểu đường/tiểu đường theo CSV UCI. Điểm đầu ra không được diễn giải thành xác suất mắc bệnh trong tương lai. Nhánh tri thức dùng TF-IDF kết hợp đồ thị khái niệm local, rồi ghép trích xuất theo mẫu; không có LLM sinh tự do hoặc Neo4j server đang vận hành. Nhánh RL chỉ học nhắc ghi nhật ký trong môi trường do tác giả định nghĩa.

Council kiểm tra hợp đồng và có quyền bác kết quả sai dạng, thiếu bằng chứng hoặc đề xuất thuốc. Đây là cơ chế kiểm tra tất định, không phải hội đồng bác sĩ hay mô hình tranh biện nhiều vòng. Các tác tử chạy trong một tiến trình Python, trao đổi đồng bộ qua MessageBus. Những lựa chọn này giữ được luồng đầu-cuối có thể quan sát và kiểm thử trong phạm vi project sinh viên.

Phần 2

Cơ sở tác tử và kiến trúc hệ thống

2.1 Tác tử và hệ đa tác tử

Trong cách tiếp cận tác tử, một thành phần quan sát môi trường, lựa chọn hành động theo mục tiêu và cung cấp phản hồi để các thành phần khác tiếp tục xử lý. Các tính chất tự chủ, phản ứng, chủ động và tương tác thường được dùng để phân tích agent [1]. Mức tự chủ của Project01 có giới hạn: tác tử có thể yêu cầu dữ liệu, từ chối khi thiếu bằng chứng hoặc trả FAILURE, nhưng không tự chọn công cụ ngoài tập được lập trình.

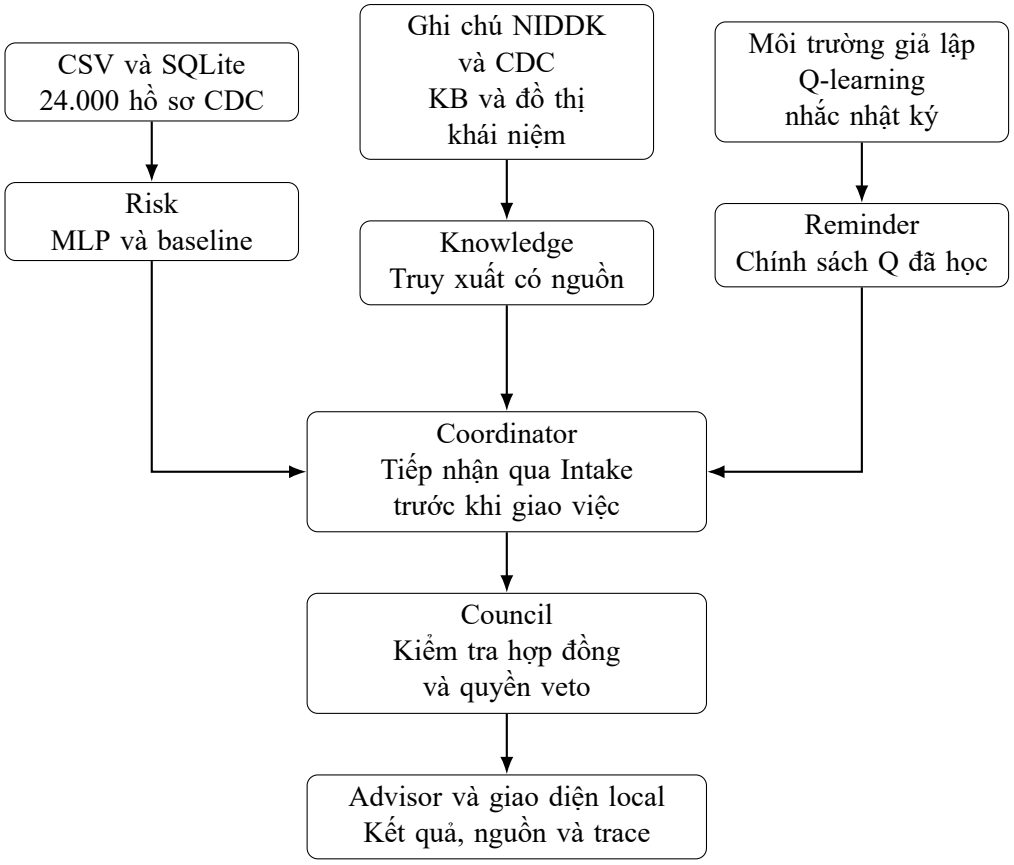
Hệ đa tác tử phân chia bài toán theo trách nhiệm và phối hợp qua một giao thức. Trong project, việc phân chia không phụ thuộc vào việc có nhiều LLM. Risk sử dụng mô hình đã học; Knowledge sử dụng kho tri thức; Reminder sử dụng Q-table; Intake và Council áp dụng luật. Các vai trò có cách ra quyết định khác nhau và được kết nối bởi Coordinator.

Bảng 2.1: Liên hệ loại tác tử và hiện thực trong Project01.

Loại cơ chế	Hiện thực và giới hạn
Phản ứng theo luật	Intake ưu tiên dấu hiệu khẩn cấp và phạm vi; chỉ bao phủ điều kiện đã mã hóa.
Dựa trên mô hình học	Risk nạp MLP hoặc baseline, suy luận trên vector khảo sát hợp lệ.
Dựa trên tri thức	Knowledge truy xuất ghi chú và liên kết khái niệm; không suy diễn lâm sàng mở.
Học từ phần thưởng	Reminder chọn argmax Q theo trạng thái mô phỏng; không tự huấn luyện trong request.
Kiểm tra và tổng hợp	Council kiểm tra hợp đồng; Advisor ghép kết quả có nguồn theo mẫu.

2.2 Kiến trúc tổng thể

Hình 2.1 thể hiện ba nhánh dữ liệu cùng hội tụ ở điều phối. Dữ liệu nghiên cứu và tri thức được chuẩn bị trước, còn mỗi request chỉ thực hiện kiểm tra, suy luận, truy xuất và tổng hợp. Không có thao tác fit trong luồng request, nên việc dự đoán của demo tương ứng với artifact đã đánh giá.



Hình 2.1: Các nguồn thông tin và cơ chế phối hợp của Diabetes Agent.

2.3 Vai trò và hợp đồng đầu ra

Bảng 2.2: Bảy vai trò tính cả điều phối.

Vai trò	Đầu vào chính	Đầu ra và trách nhiệm
Coordinator	Tác vụ, câu hỏi, hồ sơ	Giao việc theo tác vụ; dừng sớm nếu Intake không cho phép.
Intake	Câu hỏi và context	completed, urgent, needs_input hoặc out_of_scope.
Risk	21 input và model ID	Score, threshold, nhãn; liệt kê thiếu/sai mã khi chưa suy luận được.
Knowledge	Câu hỏi	Tối đa ba ghi chú, ID/URL và câu trả lời; có thể thiếu bằng chứng.
Reminder	Gap và fatigue	Trạng thái, ba Q-value và hành động nhắc việc.
Council	Các proposal	approved/rejected cùng lý do; không sửa điểm model.
Advisor	Proposal được duyệt	Ghép đoạn kết quả, nguồn và lời nhắc phạm vi.

Các tác vụ được người dùng chọn tường minh: consult, knowledge, risk hoặc reminder. Đây là chủ đích thiết kế để làm rõ hợp đồng đầu vào; project không bổ sung bộ hiểu ý định hội thoại chưa được đánh giá. consult gọi cả ba nhánh chuyên môn, trong khi các tác vụ riêng chỉ gọi thành phần cần thiết.

2.4 Thông điệp và điều kiện dừng

MessageBus nhận sender, receiver, task và payload; ghi REQUEST trước khi gọi handler, sau đó ghi INFORM hoặc FAILURE. Mỗi thông điệp phản hồi có trạng thái và thời gian xử lý. Khi handler phát sinh exception, bus chuyển lỗi thành kết quả failure để Council có thể từ chối tổng hợp. Thông tin lỗi này dùng cho demo kỹ thuật local.

Một lượt consult bình thường có sáu cặp request/response, tương ứng 12 bản ghi: Intake, Risk, Knowledge, Reminder, Council và Advisor. Nếu Intake phát hiện trạng thái urgent, hệ thống kết thúc sau hai thông điệp và không gọi Risk. Nếu Council bác proposal, Advisor không được gọi. Không có vòng lặp tranh biện vô hạn hoặc tác tử tự tạo thêm tác tử.

Bảng 2.3: Ba performative được hiện thực thực tế.

Loại	Ý nghĩa
REQUEST	Coordinator yêu cầu một handler thực hiện nhiệm vụ với payload cụ thể.
INFORM	Handler trả trạng thái và dữ liệu. approved/rejected cũng là nội dung INFORM từ Council.
FAILURE	Handler không thực hiện được do exception; chưa thể coi là kết quả thành công.

Quy ước này lấy tinh thần thông điệp tác tử nhưng không triển khai đầy đủ FIPA-ACL. Không có dịch vụ khám phá agent, transport mạng hoặc bảo đảm truyền thông phân tán. Việc gọi đồng bộ giúp kiểm tra được thứ tự và giới hạn công việc trên máy cá nhân.

2.5 Luật ưu tiên và quyền kiểm tra

Intake được thực hiện trước chuyên gia. Những cụm dấu hiệu như bất tỉnh, co giật hoặc khó thở nặng làm hệ thống chuyển nhánh cần hỗ trợ y tế. Số đo glucose bổ sung phải hữu hạn, đúng khoảng kiểm tra và được nhập theo mg/dL; dưới 70 mg/dL kích hoạt cảnh báo có nguồn CDC [10]. Đây là luật cảnh báo riêng, không phải nhãn do mô hình khảo sát suy ra.

Council kiểm tra ID bằng chứng, sự hiện diện của nguồn khi Knowledge trả completed, khoảng score và các lỗi failure. Proposal loại medication luôn bị bác. Tuy vậy, hợp đồng đúng dạng không bảo đảm câu trả lời đúng y khoa hoặc phản ánh đầy đủ tình trạng người dùng. Quyền veto là một lớp bảo vệ phần mềm có phạm vi xác định.

Phần 3

Dữ liệu và mô hình học sâu

3.1 Nguồn dữ liệu và ý nghĩa nhãn

Nguồn công khai là CDC Diabetes Health Indicators trên UCI [3]. CSV nguồn có 253.680 dòng với 21 đặc trưng và cột nhãn nhị phân. Project sử dụng mẫu nghiên cứu 24.000 dòng đã được đóng băng trong workspace trước đó. Metadata lưu cả SHA256 nguồn và SHA256 mẫu, URL và quy trình chia tập; hàm đọc dừng nếu checksum mẫu không còn khớp.

Nhãn được đổi tên từ `Diabetes_binary` thành `target`. Trong diễn giải của project, lớp 1 là nhãn gộp tiền tiểu đường/tiểu đường theo mã UCI, còn lớp 0 là không có nhãn đó trong dữ liệu. Không có căn cứ từ cột này để kết luận riêng type 2 hoặc nguy cơ mắc bệnh sau một khoảng thời gian. Trang UCI có phần giới thiệu chưa thống nhất với bảng số đặc trưng; project bám schema của CSV thực tế và ghi nhận khác biệt trong provenance.

3.2 Đặc trưng và mã hóa

Các biến bao gồm huyết áp/cholesterol cao, BMI, hút thuốc, tiền sử đột quỵ hoặc bệnh tim, vận động, rau/trái cây, bảo hiểm và rào cản chi phí, tự đánh giá sức khỏe, số ngày sức khỏe kém, khó đi lại, giới tính và các nhóm tuổi/học vấn/thu nhập.

Bảng 3.1: Nhóm đặc trưng và những điểm dễ nhập sai.

Nhóm	Mã và ý nghĩa trong project
Biến nhị phân	0/1; không tự biến câu trả lời mơ hồ thành một giá trị mặc định.
BMI	Số liên tục trong khoảng 12–98, đơn vị kg/m^2.
Age	Mã 1–13, không phải tuổi theo năm; 1 ứng với 18–24, 13 từ 80 tuổi.
GenHlth	Mã 1–5, từ sức khỏe tự đánh giá tốt tới kém.
MentHlth và PhysHlth	Số ngày 0–30 trong giai đoạn khảo sát.
Education và Income	Các nhóm 1–6 và 1–8 theo schema Hoa Kỳ; không tự quy đổi thành thước đo Việt Nam.
Cột quản trị	target, row_id, group và split không được đưa vào input.

Các mã có thứ tự được đưa vào dưới dạng số như một lựa chọn đơn giản. Điều đó áp đặt quan hệ khoảng cách cho Logistic/MLP và là một giới hạn; project chưa so sánh one-hot encoding hoặc mô hình chuyên biệt cho biến phân loại. Tất cả model dùng cùng tập đặc trưng để giữ điều kiện đối chiếu.

3.3 Chia tập và chống rò rỉ

Bảng 3.2: Phân bố tập nghiên cứu được sử dụng thực tế.

Tập	Số dòng	Nhãn 1	Tỷ lệ nhãn 1
train	16787	2375	14.15%
test	3612	496	13.73%
validation	3601	473	13.14%

Split được kế thừa nguyên từ tập nghiên cứu, với seed 42/43 ở bước chia theo nhóm. Mỗi nhóm được định nghĩa bởi fingerprint của cùng vector 21 đặc trưng. Các fingerprint giống nhau không xuất hiện ở hai tập khác nhau. Hàm `load_data` còn tính lại fingerprint để không chỉ tin cột group được lưu sẵn.

Không có định danh bệnh nhân để chứng minh mọi bản ghi của một người đều được gom cùng split. Vì vậy, phát biểu chính xác là tách nhóm vector trùng nhau, không phải đã bảo đảm tách theo người bệnh. Phân bố nhãn giữa các split có chênh lệch nhỏ do chia theo nhóm, không ép cân bằng thành 50/50.

Scaler chỉ fit trên train. Validation được dùng để chọn epoch, model mặc định và ngưỡng. Test không được dùng để chọn lại các quyết định đó. Việc chạy lại notebook sử dụng cùng source, seed và split; các lượt lặp để xác nhận tái lập không được coi là các lần thử độc lập.

3.4 MLP PyTorch và hàm mất mát

Mạng sử dụng hai lớp ẩn kích thước 64 và 32, ReLU, dropout 0,15 sau lớp đầu và một logit đầu ra. Với vector đầu vào đã chuẩn hóa x, mô hình tính:

$$h_1 = \mathrm{ReLU}(W_1 x + b_1), \tag{3.1}$$

$$h_2 = \mathrm{ReLU}(W_2\,\mathrm{Dropout}(h_1) + b_2), \tag{3.2}$$

$$z = W_3 h_2 + b_3, \qquad p = \sigma(z). \tag{3.3}$$

Số tham số học là $21 \cdot 64 + 64 + 64 \cdot 32 + 32 + 32 + 1 = 3.521$. Đầu ra logit được đưa trực tiếp vào `BCEWithLogitsLoss`; sigmoid chỉ áp dụng khi suy luận. Việc kết hợp logit và BCE trong cùng toán tử giúp ổn định tính toán theo tài liệu PyTorch [4].

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \sigma(z_i) + (1 - y_i)\log(1 - \sigma(z_i))\right]. \tag{3.4}$$

Không dùng trọng số lớp hoặc tái lấy mẫu để đổi tiên nghiệm. Dù vậy, sigmoid không được coi là xác suất lâm sàng đã hiệu chỉnh. Project chỉ báo Brier score mô tả sai số xác suất trên tập test nội bộ; chưa có calibration/external validation cho quần thể mục tiêu.

Bảng 3.3: Cấu hình huấn luyện MLP được khóa trong source.

Thông số	Giá trị
Kiến trúc	21–64–32–1
Optimizer	Adam; learning rate 0,001; weight decay 10^{-4}
Batch và epoch tối đa	256; 80
Early stopping	BCE validation; patience 10; mức cải thiện 10^{-5}
Khởi tạo	Seed 42; CPU; 2 thread PyTorch
Checkpoint	Weight có BCE validation thấp nhất, không phải epoch cuối

3.5 Các mô hình đối chứng

Logistic Regression là pipeline StandardScaler và bộ phân loại tuyến tính với tối đa 1.000 vòng lặp. Random Forest dùng 160 cây, ít nhất 8 mẫu ở lá, max_features bằng căn bậc hai và seed 42. DummyClassifier với strategy prior luôn trả phân bố nhãn train. Đây là các cấu hình cố định của project, chưa thực hiện tìm kiếm siêu tham số toàn diện.

Đối chứng giúp kiểm tra xem mạng nhiều lớp có tạo khác biệt so với cách tiếp cận đơn giản hơn trên dữ liệu bảng. Không giả định học sâu luôn có ưu thế. Model mặc định được chọn bằng Average Precision validation, và có thể là Random Forest dù MLP là thành phần học sâu cần trình bày.

3.6 Ngưỡng và chỉ số đánh giá

Mỗi model chọn ngưỡng từ 151 ứng viên đều nhau trong khoảng 0,05–0,80, tối đa hóa F1 validation; nếu bằng điểm, chọn ứng viên xuất hiện đầu tiên. Sau đó ngưỡng được giữ nguyên khi đánh giá test. Cách này phục vụ benchmark kỹ thuật; không tối ưu chi phí lâm sàng của âm tính giả/dương tính giả.

ROC AUC mô tả khả năng xếp hạng giữa hai lớp. Average Precision tổng hợp precision theo các mức recall và phù hợp để quan sát bài toán lớp dương ít [5]. Precision, recall, specificity, F1 và ma trận nhầm lẫn được báo tại ngưỡng đã chọn. Brier là trung bình $(p_i - y_i)^2$, thấp hơn là tốt hơn trong cùng điều kiện dữ liệu.

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{3.5}$$

$$\text{F1} = \frac{2PR}{P + R}, \qquad \text{Specificity} = \frac{TN}{TN + FP}. \tag{3.6}$$

Accuracy không được dùng làm kết luận chính vì phần lớn mẫu thuộc lớp 0. Đường chuẩn Average Precision của bộ dự đoán không có khả năng phân biệt bằng tỷ lệ lớp 1 trong tập đánh giá.

Phần 4

Tri thức và học tăng cường

4.1 Quy trình xây dựng kho tri thức

Kho tri thức gồm 12 ghi chú tiếng Việt biên soạn ngắn từ bảy trang chính thức NIDDK và CDC. Mỗi ghi chú lưu ID, tiêu đề, từ khóa, khái niệm, nội dung, tên nguồn, URL và ngày kiểm tra. Đây là các đoạn giải thích phục vụ học tập, không phải bản sao đầy đủ của tài liệu hướng dẫn hoặc kết quả crawl tự động toàn bộ website.

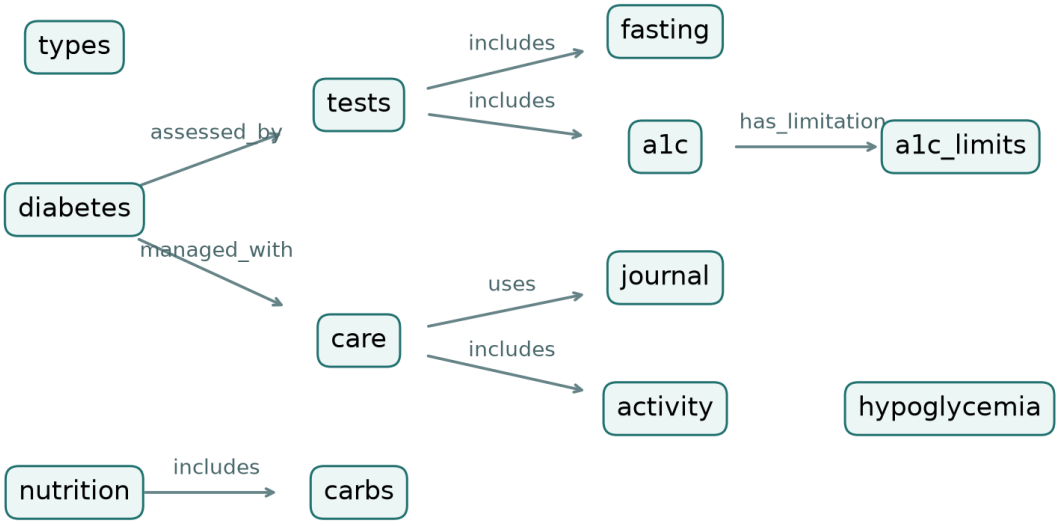
Quy trình gồm chọn chủ đề có nguồn, đọc nội dung chính thức, viết đoạn tóm lược theo phạm vi project, gán concept và liên kết nguồn, sau đó đánh giá khả năng tìm lại đoạn tương ứng. Nội dung không được thêm số liệu điều trị không xuất hiện trong nguồn. Các nguồn về xét nghiệm, HbA1c, quản lý bệnh, dinh dưỡng và vận động được liệt kê trong tài liệu tham khảo [7, 8, 9, 11, 12].

Bảng 4.1: Danh mục 12 ghi chú có liên kết tới bảy trang nguồn.

ID	Chủ đề	Nguồn
D01	Tiểu đường và insulin	NIDDK
D02	Các loại tiểu đường	NIDDK
D03	Xét nghiệm và xác nhận	NIDDK
D04	Glucose lúc đói	NIDDK
D05	Đường huyết thấp	CDC
D06	HbA1c phản ánh thời gian dài	NIDDK
D07	Giới hạn của HbA1c	NIDDK
D08	Nhật ký theo dõi	NIDDK
D09	Mục tiêu và thuốc	NIDDK
D10	Ăn uống và khẩu phần	CDC
D11	Carbohydrate	CDC
D12	Vận động	CDC

4.2 Đồ thị khái niệm local

Đồ thị có 12 khái niệm, 8 quan hệ có hướng giữa khái niệm và 12 cạnh nối ghi chú tới khái niệm được giải thích. Ví dụ, tests nối tới fasting và a1c; a1c nối tới a1c_limits. Cấu trúc này làm tường minh một số quan hệ giữa các phần tri thức và cung cấp một bước mở rộng khi truy xuất.



Hình 4.1: Đồ thị 12 khái niệm và các quan hệ được biên soạn trong project. Các ghi chú D01–D12 được gắn cạnh bằng chứng trong JSON.

Graph được lưu bằng JSON và nạp trực tiếp khi ứng dụng khởi động. File `concept_graph.cypher` xuất nút/cạnh khái niệm để tham khảo nếu muốn tiếp tục với Neo4j; bản local không yêu cầu cài Neo4j và chưa được đánh giá trên một Neo4j server. Không gọi cách mở rộng nhỏ này là triển khai đầy đủ các phương pháp GraphRAG quy mô lớn.

4.3 Truy xuất và kết xuất câu trả lời

Văn bản được chuẩn hóa chữ thường và bỏ dấu, sau đó vector hóa TF-IDF với unigram/bigram. Corpus đầu vào cho từng ghi chú gồm tiêu đề, từ khóa và nội dung. Điểm lexical là cosine similarity giữa query và mỗi ghi chú. Alias khớp theo ranh giới từ giúp xác định các concept được nhắc trực tiếp.

Khi bật đồ thị, hệ thống mở rộng một bước theo cạnh có hướng. Ghi chú thuộc các concept trực tiếp hoặc mở rộng được cộng 0,12 vào điểm lexical. Để tránh suy ra bằng chứng chỉ vì ở gần trong graph, ứng viên phải có cosine ít nhất 0,10 nếu không khớp concept trực tiếp. Ngoài ra, ứng viên cần khớp alias hoặc có ít nhất hai token query ở tiêu đề/từ khóa sau khi bỏ một số từ chức năng.

$$\text{score}(d, q) = \cos(v_d, v_q) + 0,12 \mathbb{1}\{C_d \cap C_q^+ \neq \varnothing\}. \tag{4.1}$$

Các điều kiện trên là heuristic được định nghĩa trong source; chưa có tập phát triển độc lập để chứng minh ngưỡng tối ưu. Chúng ngăn một số kết quả do trùng từ thông thường nhưng cũng có thể làm bỏ sót cách diễn đạt mới.

Tối đa ba ghi chú được trả về theo điểm, nếu bằng điểm thì theo ID. Không có ứng viên đạt thì trả insufficient_evidence. Câu trả lời nối trực tiếp nội dung ghi chú cùng ID, còn giao diện hiển thị URL tương ứng. Đây là RAG theo kiểu truy xuất và kết xuất trích đoạn theo mẫu, không dùng LLM hoặc embedding model. Council kiểm tra tính hợp lệ của ID nhưng không chứng minh độ liên quan ngữ nghĩa của từng đoạn.

4.4 Thiết kế đánh giá truy xuất

Bộ đánh giá có 24 câu do tác giả soạn, mỗi ghi chú hai câu và một ID đích. Hit@1 kiểm tra kết quả đầu; Hit@3 kiểm tra ID xuất hiện trong ba kết quả; MRR lấy trung bình nghịch đảo thứ hạng, bằng 0 nếu không tìm thấy. Cùng bộ câu được chạy có và không có bonus đồ thị để xem đóng góp của thành phần này.

Bộ câu nhỏ và dùng từ vựng gần KB, nên không đại diện câu hỏi người bệnh ngoài thực tế. Các bài kiểm tra đã giúp phát hiện lỗi trong quá trình phát triển; vì vậy kết quả cuối là bằng chứng hồi quy và chức năng, không phải một test truy xuất độc lập chưa từng được xem. Báo cáo không dùng điểm cao để tuyên bố khả năng hiểu hội thoại y tế tổng quát.

4.5 MDP nhắc ghi nhật ký

Nhánh RL minh họa cách tác tử cải thiện chính sách qua phản hồi. Mục tiêu là cân bằng việc có bản ghi với chi phí và cảm giác mệt do thông báo. Mọi hệ số phản ứng/phần thưởng dưới đây do tác giả đặt để tạo môi trường học, không được ước lượng từ bệnh nhân hoặc thử nghiệm y khoa.

Trạng thái có dạng $s = 2g + f$, trong đó $g \in \{0, 1, 2, 3\}$ là số ngày chưa ghi, được chặn tại 3, và $f \in \{0, 1\}$ là trạng thái mệt vì thông báo. Có 8 trạng thái và 3 hành động: $a = 0$ không nhắc, $a = 1$ nhắc ghi nhật ký, $a = 2$ gửi gợi ý ghi chép ngắn. Reminder chỉ quyết định hành động trong tập này.

Xác suất ghi nhận ở bước tiếp theo được mô phỏng:

$$P(L = 1 \mid g, f, a) = \text{clip}(0,40 - 0,07g - 0,08f + b_a;\ 0,05;\ 0,95), \tag{4.2}$$

với $b_a \in \{0; 0,35; 0,22\}$. Nếu có bản ghi, $g' = 0$; nếu không, $g' = \min(3, g + 1)$. Xác suất mệt ở ngày tiếp theo là 0,15; 0,70 hoặc 0,40 theo hành động tương ứng. Reward:

$$r = L - c_a - 0,10g' - 0,12f\mathbb{1}\{a > 0\}, \qquad c_a \in \{0; 0,22; 0,10\}. \tag{4.3}$$

Một policy có reward cao hơn chỉ phù hợp hơn với utility của simulator này. Reward không có đơn vị đường huyết và không đo mức cải thiện sức khỏe.

4.6 Huấn luyện Q-learning

Theo quy tắc Q-learning [2], tác tử cập nhật:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]. \tag{4.4}$$

Project dùng $\alpha = 0,08$, $\gamma = 0,92$, ϵ-greedy với $\epsilon_e = \max(0,03; 0,8 \cdot 0,997^e)$. Mỗi seed train có 1.800 episode, mỗi rollout 28 ngày. Đây là bài toán continuing: mốc 28 ngày chỉ là cửa sổ lấy mẫu, không phải trạng thái bệnh kết thúc, nên bước cuối vẫn bootstrap từ Q của trạng thái kế.

Năm seed train là 42–46. Mỗi policy được đánh giá greedy bằng 200 seed 10000–10199, tách khỏi seed train. Các policy đối chứng gồm không bao giờ nhắc, luôn nhắc và luật chỉ nhắc khi gap từ 2 và chưa mệt. Các lượt eval dùng cùng seed giữa policy để giảm nhiễu so sánh.

Q-table của seed 42 được dùng trong demo theo quy định trước. Project không chọn seed đạt điểm cao nhất. Độ lệch chuẩn trong bảng tổng hợp là độ biến thiên giữa năm reward trung bình của các seed train, không phải khoảng tin cậy lâm sàng.

4.7 Kết hợp luật với chính sách

Intake có thể dừng toàn bộ request trước khi Reminder được gọi. Council bác các proposal medication; action space của Reminder cũng không có hành động dùng thuốc. Khi gap/fatigue sai mã, Reminder trả needs_input thay vì tự cắt giá trị vào miền hợp lệ. Như vậy, kết quả học được không được dùng để vượt qua giới hạn đầu vào và phạm vi hệ thống.

Project chưa học policy từ dữ liệu hành vi thật, chưa kiểm tra offline policy evaluation, chưa mô hình hóa sở thích cá nhân hoặc ràng buộc dài hạn. Việc mở rộng chỉ có ý nghĩa khi có dữ liệu và thiết kế nghiên cứu phù hợp; không suy rộng từ bảng Q nhỏ sang chính sách chăm sóc thực tế.

Phần 5

Mã nguồn và notebook thực hành

5.1 Cấu trúc thư mục

Mã được tổ chức theo chức năng để dễ đọc và tránh hai phiên bản thuật toán khác nhau trong notebook và demo. Notebook giải thích quá trình, gọi fit/evaluate thật và lưu output. Module chứa implementation dùng chung, còn app.py chỉ xây giao diện và gọi hệ thống.

```
PROJECT_01/
  app.py
  run.cmd
  requirements.txt
  diabetes_agent/
    config.py          data.py           models.py
    knowledge.py       rl.py             system.py
    evaluation.py
  notebooks/           # 01 -> 04, executed outputs
  data/                # CSV, provenance, KB, graph, cases
  artifacts/           # models, metrics, predictions, Q, audit
  tests/test_project.py
  scripts/
    run_experiments.py    execute_notebooks.py
    update_report_assets.py
  docs/                # local run and demo instructions
  report/              # PDF, LaTeX, measured tables, images
```

Mã nguồn 5.1: Cấu trúc các thành phần bàn giao.

5.2 Các hàm và lớp chính

Bảng 5.1: Đường đi từ yêu cầu tới source code.

Module	Hàm hoặc lớp	Vai trò
data.py	load_data, read_database	Kiểm tra checksum/split, dựng SQLite và đọc bảng nghiên cứu.
models.py	train_all, RiskModel	Train/checkpoint/threshold/test; nạp artifact để suy luận.
knowledge.py	KnowledgeBase	TF-IDF, concept expansion, bằng chứng và trả lời trích xuất.
rl.py	transition, train_q	Mô phỏng một bước và cập nhật Q; xuất lịch sử/đối chứng.
system.py	MessageBus, DiabetesSystem	Hợp đồng thông điệp, điều phối, tổng hợp và audit SQLite.
evaluation.py	evaluate_all	Bộ truy xuất, kịch bản hệ thống và inject lỗi Council.

5.3 Notebook 01 từ dữ liệu tới model đã lưu

Notebook đầu tiên kiểm tra nguồn, hiển thị split và dữ liệu, tạo SQLite, xem kiến trúc rồi thực thi `train_all`. Vòng lặp minibatch ở module thực hiện forward, loss, backward và optimizer step. Sau mỗi epoch, mạng chuyển sang eval để tính BCE validation, không bật dropout. Weight tốt nhất được sao chép độc lập trước khi những epoch sau thay đổi model.

```
In [5]
# Fit thật, lưu weight/scaler, quyết định validation, rồi xuất test predictions.
from diabetes_agent.models import train_all
test_metrics, manifest = train_all()
display(pd.read_csv(ARTIFACTS / "metrics_validation.csv"))
print("Mặc định đã chọn bằng validation:", manifest["default_model"])
print("Epoch MLP được chọn:", manifest["best_epoch"])

Output
           model   roc_auc  average_precision        f1  precision    recall  \
0          dummy  0.500000           0.131352  0.232204   0.131352  1.000000
1       logistic  0.812739           0.348209  0.434981   0.339667  0.604651
2  random_forest  0.810360           0.371311  0.438662   0.338303  0.623679
3            mlp  0.816985           0.370597  0.441360   0.328512  0.672304

   specificity     brier  threshold    tn    fp   fn   tp
0     0.000000  0.114201      0.050     0  3128    0  473
1     0.822251  0.098623      0.215  2572   556  187  286
2     0.815537  0.096658      0.230  2551   577  178  295
3     0.792199  0.095996      0.205  2478   650  155  318
Mặc định đã chọn bằng validation: random_forest
Epoch MLP được chọn: 21
```

Hình 5.1: Cell huấn luyện thật trong notebook 01 và một phần output đã lưu. Nội dung ảnh được dựng từ source/output của ipynb đã thực thi.

Weight MLP lưu bằng state_dict, scaler lưu bằng joblib. Khi nạp lại, chương trình dựng cùng kiến trúc rồi load weight ở CPU với weights_only. Baseline sklearn được lưu riêng. Không nạp artifact joblib từ nguồn không tin cậy; trong gói này chúng được tạo bởi source của chính project.

Cell cuối đối chiếu một score suy luận từ weight đã nạp với prediction CSV của cùng row_id trên test, sai lệch cho phép 10^{-6}. Kiểm tra này xác nhận tính nhất quán của đường lưu/nạp và inference, không bổ sung bằng chứng độc lập về khả năng tổng quát hóa.

5.4 Notebook 02 từ query tới nguồn

Notebook thứ hai liệt kê 12 ghi chú và quan hệ, minh họa query HbA1c và thiếu máu, hiển thị lexical score, score sau graph và matched_concepts. Cell sau kiểm tra câu hỏi ngoài chủ đề và cách viết không dấu. Cuối notebook là bảng 24 câu có/không có graph và source hàm truy xuất.

```
In [3]
query = "HbA1c và thiếu máu"
hits = kb.retrieve(query)
display(pd.DataFrame(hits)[["id", "title", "lexical_score", "score",
"matched_concepts"]])
print(kb.answer(query)["text"])
for item in hits:
    print(item["id"], item["source_url"])

Output
    id                          title  lexical_score     score matched_concepts
0  D07            Giới hạn của HbA1c       0.309929  0.429929     [a1c_limits]
1  D06  HbA1c phản ánh thời gian dài       0.134940  0.254940            [a1c]
Một số tình trạng ảnh hưởng hồng cầu hoặc hemoglobin có thể làm HbA1c kém tin cậy.
Khi kết quả không phù hợp, cần trao đổi với nhân viên y tế. [D07]

HbA1c phản ánh đường huyết trung bình khoảng ba tháng, khác số đo tức thời. Cần đọc
kết quả cùng bối cảnh và hướng dẫn chuyên môn. [D06]
D07 https://www.niddk.nih.gov/health-information/diagnostic-tests/a1c-test
D06 https://www.niddk.nih.gov/health-information/diagnostic-tests/a1c-test
```

Hình 5.2: Ví dụ truy xuất trong notebook 02; ID nguồn gắn với đoạn được trả về.

Để tái lập, người đọc có thể sửa query của cell minh họa mà không thay bộ câu đánh giá. Nếu thay thuật toán hoặc KB sau khi xem kết quả bộ câu, cần xem đây là kiểm tra trong phát triển và bổ sung bộ đánh giá độc lập khi muốn tuyên bố hiệu năng.

5.5 Notebook 03 học và kiểm tra policy

Notebook thứ ba in trực tiếp hàm transition để người đọc nhìn thấy các giả định về phản ứng. Cell train thực hiện năm seed, xuất Q-table seed 42, history và bảng đối chứng. Các Q-value được đặt cạnh gap/fatigue, rồi giải mã argmax thành tên hành động.

```
In [4]
import numpy as np
q = np.load(ARTIFACTS / "q_table.npy")
policy = pd.DataFrame(q, columns=ACTIONS)
policy.insert(0, "gap_days", np.arange(8)//2)
policy.insert(1, "fatigue", np.arange(8)%2)
policy["chosen_action"] = [ACTIONS[i] for i in q.argmax(axis=1)]
display(policy.round(3))

Output
   gap_days  fatigue  Không nhắc  Nhắc ghi nhật ký  Gửi gợi ý ghi chép ngắn  \
0         0        0       3.523             3.823                    3.516
1         0        1       3.638             3.101                    3.187
2         1        0       3.268             3.293                    3.465
3         1        1       3.025             3.077                    3.146
4         2        0       2.977             3.160                    3.401
5         2        1       3.242             2.881                    2.925
6         3        0       2.651             2.962                    2.819
7         3        1       2.745             2.730                    3.060

             chosen_action
0         Nhắc ghi nhật ký
1              Không nhắc
2  Gửi gợi ý ghi chép ngắn
3  Gửi gợi ý ghi chép ngắn
4  Gửi gợi ý ghi chép ngắn
5              Không nhắc
6         Nhắc ghi nhật ký
7  Gửi gợi ý ghi chép ngắn
```

Hình 5.3: Cell đọc Q-table thực tế của notebook 03 và chính sách cho tám trạng thái.

Một phép kiểm tra riêng chạy hai lượt cùng seed với ít episode và xác nhận hai Q-table bằng nhau. Đây là kiểm tra bộ sinh ngẫu nhiên và luồng cập nhật, không phải kiểm tra Q-learning đã hội tụ tới tối ưu. Project không đưa ra chứng minh hội tụ thực nghiệm chỉ dựa trên đường học hữu hạn.

5.6 Notebook 04 tích hợp và kiểm tra veto

Notebook cuối nạp hệ thống, sử dụng hồ sơ minh họa thuộc train, chạy consult và hiển thị 12 thông điệp. Tiếp đó là hồ sơ thiếu chỉ có BMI, số đo glucose thấp, proposal thuốc và ID bằng chứng giả. Người đọc có thể quan sát trạng thái needs_input, urgent và rejected theo từng tình huống.

```
In [5]
urgent = system.run("Tôi hỏi về HbA1c", profile, context={"glucose_mg_dl": 62})
print(urgent["status"], urgent["text"])
display(pd.DataFrame(urgent["trace"]))
assert len(urgent["trace"]) == 2
assert not any(m["receiver"] == "Risk" for m in urgent["trace"])

Output
urgent Số đo dưới 70 mg/dL cần được xử trí kịp thời theo hướng dẫn của nhóm chăm
sóc. Liên hệ hỗ trợ y tế; nếu không tỉnh táo hoặc không thể tự xử trí, cần trợ giúp
khẩn cấp.

        sender     receiver performative         task  status  elapsed_ms
0  Coordinator       Intake      REQUEST  CHECK_SCOPE     NaN         NaN
1       Intake  Coordinator       INFORM  CHECK_SCOPE  urgent       0.037
```

Hình 5.4: Kiểm tra nhánh urgent trong notebook 04; trace cho thấy chưa gọi mô hình Risk.

Cuối notebook chạy lại bộ đánh giá và assert toàn bộ kịch bản đạt. Script `execute_notebooks.py` khởi tạo kernel bằng đúng Python gọi script, chạy tuần tự trong các tiến trình riêng để cô lập vòng đời kernel trên Windows, lưu output vào ipynb, kiểm tra execution count và lỗi cell, rồi đóng kernel. Không tạo output mẫu thay cho việc thực thi.

5.7 Lưu trữ và chế độ local

CSDL `research.sqlite3` được dựng lại từ CSV nghiên cứu. Audit demo là database khác; mặc định ứng dụng không tự lưu câu hỏi hoặc đầu vào người dùng. Khi người trình diễn chủ động chọn lưu, câu hỏi, câu trả lời và trace được ghi tại `demo_audit.sqlite3`. Nội dung câu hỏi vẫn có thể chứa thông tin người dùng nhập nên không dùng hồ sơ thật trong lớp học.

SQLite dùng placeholder để ghi JSON và khóa run_id. Không có đăng nhập, phân quyền, mã hóa database hoặc quy trình vận hành cho dữ liệu bệnh nhân. Server bind localhost và ứng dụng không gửi nội dung tới dịch vụ mô hình bên ngoài. Các URL nguồn chỉ để người dùng mở khi cần đọc tài liệu gốc.

Phần 6

Thực nghiệm và phân tích kết quả

6.1 Điều kiện thực nghiệm

Các thực nghiệm chạy trên CPU với Python 3.11.9, PyTorch 2.15.0+cpu, scikit-learn 1.9.1, pandas 2.3.3 và Streamlit 1.66.1. Phiên bản trực tiếp được ghi trong requirements, còn môi trường kiểm tra đầy đủ nằm ở `artifacts/environment.txt`. Môi trường cài mới trên máy khác chưa được kiểm chứng; việc bàn giao xác nhận source và artifact chạy trong môi trường nêu trên.

Dữ liệu supervised dùng một split/seed cố định. RL chạy năm seed train và các seed eval tách biệt. Đánh giá phần mềm và truy xuất sử dụng kịch bản do tác giả xây. Mỗi loại bằng chứng trả lời một câu hỏi khác nhau: test supervised đo phân loại nội bộ, simulator đo utility giả định, bộ tình huống đo tuân thủ hợp đồng. Không gộp các điểm này thành một tỷ lệ chính xác chung của hệ thống y tế.

6.2 Kết quả validation và lựa chọn mặc định

Bảng 6.1: Chỉ số trên validation; ngưỡng được chọn bằng F1 validation.

Model	AUC	AP	F1	Precision	Recall	Brier
dummy	0.5000	0.1314	0.2322	0.1314	1.0000	0.1142
logistic	0.8127	0.3482	0.4350	0.3397	0.6047	0.0986
random_forest	0.8104	0.3713	0.4387	0.3383	0.6237	0.0967
mlp	0.8170	0.3706	0.4414	0.3285	0.6723	0.0960

Random Forest có Average Precision validation khoảng 0,3713, nhỉnh hơn MLP khoảng 0,3706. Theo quy tắc đã đặt, Random Forest được chọn làm model mặc định. MLP có ROC AUC validation cao hơn, nhưng tiêu chí chọn đã khóa là AP nên không đổi sang AUC sau khi thấy bảng kết quả.

Chênh lệch AP giữa hai model rất nhỏ và chưa có phân tích độ bất định để kết luận model nào tốt hơn ổn định. Lựa chọn mặc định chỉ là kết quả áp dụng quy tắc trong lần chia tập này. Giao diện vẫn cho chọn MLP để trình diễn nhánh học sâu và so sánh với các baseline.

6.3 Kết quả trên test đã khóa

Bảng 6.2: Chỉ số trên test; ngưỡng được chọn bằng F1 validation.

Model	AUC	AP	F1	Precision	Recall	Brier
dummy	0.5000	0.1373	0.2415	0.1373	1.0000	0.1185
logistic	0.8118	0.3820	0.4367	0.3499	0.5806	0.1003
random_forest	0.8096	0.3906	0.4534	0.3564	0.6230	0.0993
mlp	0.8168	0.3916	0.4431	0.3358	0.6512	0.0987

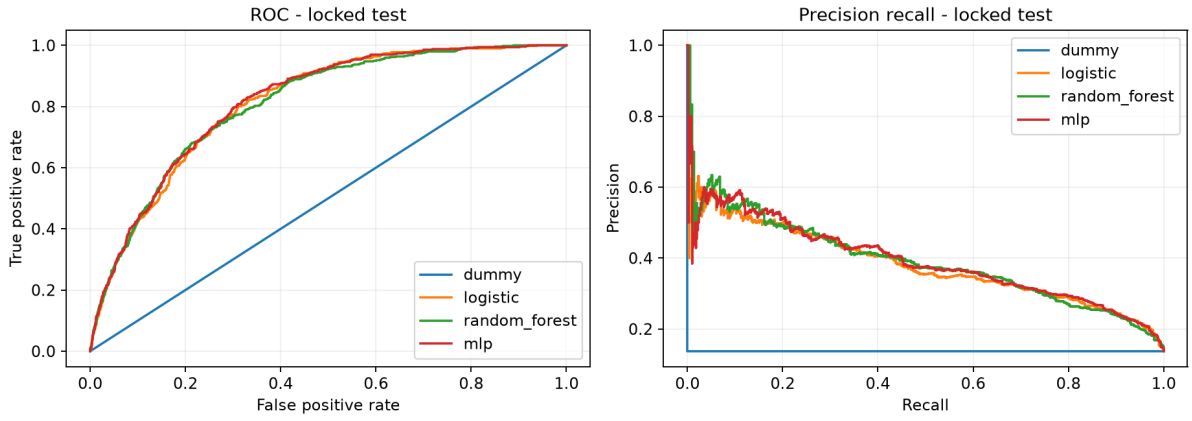
Bảng 6.3: Ngưỡng đã khóa và số lượng dự đoán trên test.

Model	Ngưỡng	TN	FP	FN	TP	Specificity
dummy	0.050	0	3116	0	496	0.0000
logistic	0.215	2581	535	208	288	0.8283
random_forest	0.230	2558	558	187	309	0.8209
mlp	0.205	2477	639	173	323	0.7949

Quan sát. MLP đạt ROC AUC khoảng 0,8168 và AP khoảng 0,3916; Random Forest đạt AUC khoảng 0,8096 và AP khoảng 0,3906. Logistic Regression đạt AUC khoảng 0,8118. Các model học được đều có AP cao hơn Dummy khoảng 0,1373, là tỷ lệ lớp dương trong test.

Diễn giải. MLP nhỉnh hơn về AUC/AP trên test nhưng không làm thay đổi quyết định mặc định đã khóa từ validation. Các chênh lệch này là mô tả trên cùng split, chưa đủ để khẳng định ưu thế tổng quát. Mô hình bảng đơn giản hơn vẫn là đối chứng cần thiết cho một mạng hai lớp ẩn.

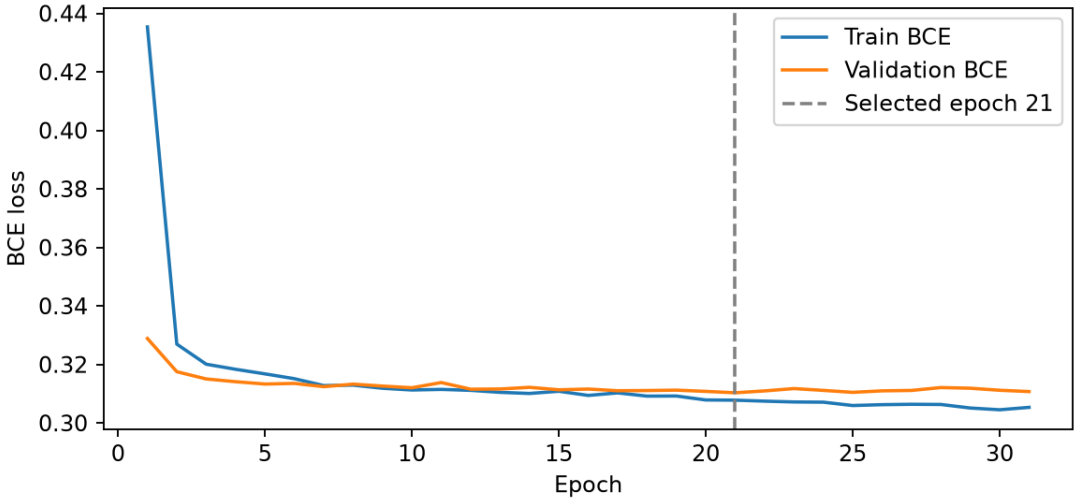
Đánh đổi tại ngưỡng. MLP phát hiện 323/496 mẫu lớp 1, còn bỏ sót 173 mẫu và tạo 639 dương tính giả. Random Forest phát hiện 309 mẫu, bỏ sót 187 và tạo 558 dương tính giả. Khi tăng recall, precision có thể giảm; việc chọn F1 validation không đồng nghĩa chọn ngưỡng phù hợp cho quyết định lâm sàng.



Hình 6.1: ROC và Precision–Recall trên test. Mọi quyết định về model/ngưỡng được thực hiện bằng validation trước đó.

Dummy được chọn ngưỡng nhỏ hơn điểm tiên nghiệm nên dự đoán tất cả là lớp 1, dẫn tới recall 1 và specificity 0. Điều này minh họa vì sao một chỉ số riêng lẻ có thể gây hiểu nhầm. Cần đọc đồng thời precision, recall và số lượng FP/FN.

6.4 Quá trình học của MLP



Hình 6.2: Loss train và validation, đường dọc thể hiện epoch được chọn để lưu checkpoint.

Checkpoint MLP được chọn tại epoch 21 theo BCE validation. Train tiếp tục cho tới khi đủ số epoch không cải thiện rồi nạp lại checkpoint tốt nhất. Train loss và validation loss không nhất thiết cùng giảm ở các epoch cuối; đó là lý do giữ checkpoint dựa trên tập không tham gia cập nhật gradient.

Brier test của MLP khoảng 0,0987, thấp hơn Dummy khoảng 0,1185. Đây là dấu hiệu mô hình cung cấp score hữu ích hơn tiên nghiệm trong điều kiện test nội bộ. Chưa có đường calibration theo nhóm, khoảng tin cậy hoặc đánh giá dịch chuyển phân bố nên không thể chuyển score đó thành nguy cơ cá nhân ở Việt Nam.

6.5 Truy xuất tri thức và vai trò đồ thị

Bảng 6.4: Truy xuất trên 24 câu do tác giả thiết kế; không phải kiểm định bên ngoài.

Cấu hình	Hit@1	Hit@3	MRR
TF-IDF	1.0000	1.0000	1.0000
TF-IDF + graph	1.0000	1.0000	1.0000

Cả hai cấu hình đều tìm đúng ID đầu tiên ở toàn bộ 24 câu trong phiên bản cuối. Vì vậy, bộ câu hiện tại không cho thấy lợi ích đo được của bonus đồ thị về Hit@1, Hit@3 hoặc MRR. Đồ thị

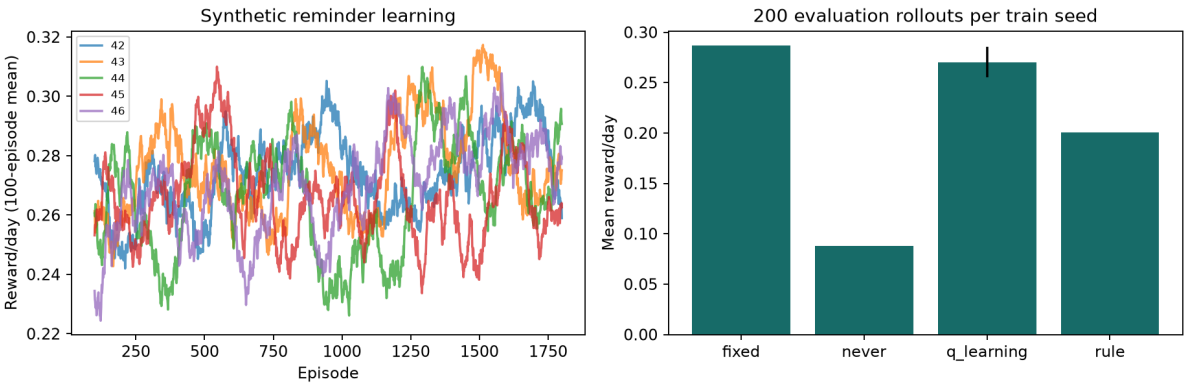
vẫn có chức năng biểu diễn liên kết và mở rộng concept trong implementation, nhưng chưa có bằng chứng nó làm tăng chất lượng trên tập kiểm tra này.

Điểm tuyệt đối cần được đọc cùng giới hạn thiết kế: chỉ 12 ghi chú, câu hỏi do chính tác giả viết và dùng lại để kiểm tra trong phát triển. Không có bộ paraphrase độc lập hoặc đánh giá từ người dùng. Một bước tiếp theo có giá trị là thu thập câu hỏi ngoài từ vựng KB, nhiều chủ đề và trường hợp cần từ chối, rồi khóa bộ test riêng.

6.6 Kết quả Q-learning và đối chứng

Bảng 6.5: Reward mỗi ngày trong simulator; std giữa năm giá trị trung bình theo seed train.

Policy	Reward trung bình	Độ lệch chuẩn
fixed	0.2868	0.0000
never	0.0876	0.0000
q_learning	0.2703	0.0153
rule	0.2006	0.0000



Hình 6.3: Đường học năm seed và reward đánh giá trong môi trường nhắc việc giả lập.

Quan sát. Q-learning có reward trung bình khoảng 0,2703 mỗi ngày, cao hơn never khoảng 0,0876 và rule khoảng 0,2006, nhưng thấp hơn fixed khoảng 0,2868. Độ lệch chuẩn giữa năm seed train của Q-learning khoảng 0,0153.

Diễn giải. Trong simulator này, lợi ích ghi nhận do hành động nhắc thường bù được chi phí nên policy luôn nhắc là đối chứng mạnh. Q-learning hữu hạn với learning rate cố định và năm seed chưa vượt đối chứng đó. Kết quả không được sửa bằng cách đổi reward sau khi xem điểm; báo cáo giữ nguyên cả trường hợp RL không tốt nhất.

Giới hạn của phép đo. Fixed/never/rule có std giữa seed train bằng 0 vì cùng policy được đánh giá lại với cùng 200 seed eval, không phải vì mọi rollout có reward giống nhau. Các dòng lặp của đối chứng không tạo thêm mẫu độc lập. Q-table seed 42 được nạp trong demo theo quy ước trước, không chọn theo reward eval.

6.7 Kiểm tra tích hợp và lỗi được inject

Bảng 6.6: Các kịch bản chức năng được chạy ở phiên bản bàn giao.

Kịch bản	Trạng thái mong đợi	Đạt	Thông điệp
risk_complete	completed	Có	8
risk_mlp	completed	Có	8
risk_missing	needs_input	Có	8
risk_invalid	needs_input	Có	8
risk_nan	needs_input	Có	8
knowledge_a1c	completed	Có	8
knowledge_food	completed	Có	8
knowledge_activity	completed	Có	8
knowledge_unknown	insufficient_evidence	Có	8
knowledge_blank	insufficient_evidence	Có	8
urgent_text	urgent	Có	2
urgent_seizure	urgent	Có	2
urgent_glucose	urgent	Có	2
invalid_unit_value	needs_input	Có	2
medication	out_of_scope	Có	2
reminder	completed	Có	8
invalid_reminder	needs_input	Có	8
consult	completed	Có	12
consult_missing	needs_input	Có	12
consult_unknown	insufficient_evidence	Có	12

Toàn bộ 20 kịch bản đạt trạng thái mong đợi ở phiên bản bàn giao. Kịch bản phủ luồng dự đoán, dữ liệu thiếu/sai mã, query có/không có bằng chứng, số đo bổ sung không hợp lệ, nhánh khẩn cấp, phạm vi thuốc và reminder. Đây là tỷ lệ đạt bộ yêu cầu phần mềm hữu hạn, không phải tỷ lệ tư vấn đúng trong mọi tình huống.

Sáu proposal lỗi được inject riêng gồm thuốc, tri thức không nguồn, ID giả, score vượt khoảng, score NaN và lỗi handler. Council bác cả sáu. Ở cấu hình bỏ Council, các proposal được đánh dấu unreviewed; không gọi đó là đã qua kiểm tra hoặc đã gây hại. Phép ablation xác nhận chức năng veto đối với lỗi được dựng sẵn, chưa chứng minh MAS tốt hơn một pipeline đơn tác tử dùng cùng luật và công cụ.

6.8 Kiểm thử tự động và notebook

Bộ pytest có 32 trường hợp và đều đạt, bao gồm tính toàn vẹn dữ liệu, mã khảo sát, nạp model, truy xuất, ưu tiên urgent, Council veto, cô lập lỗi, Q-learning và lưu audit. Kiểm thử giao diện bằng Streamlit AppTest chạy hội chẩn, ba kịch bản nhanh, nhập JSON thiếu trường và xử lý glucose thấp.

Bốn notebook có lần lượt 7, 6, 5 và 8 code cell, tổng cộng 26 cell. Tất cả có execution count và không có error output. Log lưu tên file, số cell và thời gian thực thi. Việc kiểm tra trực tiếp bằng trình duyệt còn được dùng để quan sát màu chữ, bố cục và ảnh minh chứng, bổ sung cho kiểm thử cấu trúc bằng AppTest.

Phần 7

Chạy local và trình diễn hệ thống

7.1 Cài đặt môi trường

Người nhận giải nén toàn bộ thư mục Project01, cài Python 3.11 64-bit và mở terminal tại nơi có `app.py`. Mô hình, scaler, dữ liệu và KB đã được đóng gói; lần cài đầu chỉ cần tải thư viện. Sau đó demo suy luận có thể hoạt động không cần Internet hoặc API key.

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install torch==2.15.0 --index-url https
    ://download.pytorch.org/whl/cpu
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m streamlit run app.py --server.address
    127.0.0.1 --server.port 8521
```

Mã nguồn 7.1: Tạo môi trường và khởi động bằng PowerShell.

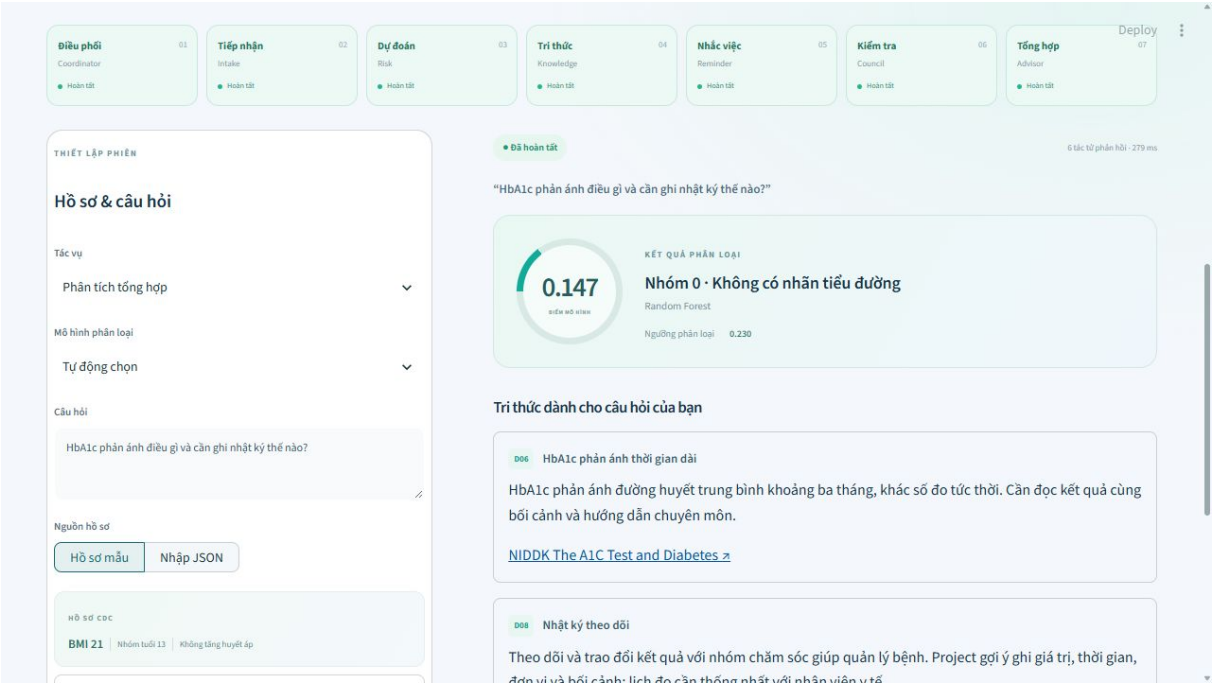
Mở trình duyệt tại http://127.0.0.1:8521. Dừng server bằng Ctrl+C trong terminal. Có thể bấm đúp `run.cmd` để tự tạo môi trường và cài các thư viện ở lần đầu; script cần Python launcher có thể tìm Python 3.11. Nếu dùng Anaconda, tạo env Python 3.11, activate rồi gọi `python -m pip` và `python -m streamlit` với cùng tham số.

Gọi trực tiếp Python trong venv giúp tránh lỗi chính sách PowerShell đối với Activate.ps1. Server chỉ bind localhost; project chưa có cấu hình triển khai công khai, đăng nhập hay phân quyền. Khi port 8521 đang được dùng, đổi port và mở đúng URL mới.

7.2 Các tab và luồng thao tác

Giao diện dashboard có bốn tab. Hội chẩn demo có ba nút chạy nhanh, bảy thẻ trạng thái tác tử và vùng thiết lập hồ sơ/câu hỏi. Kết quả chia thành vòng điểm phân loại, các thẻ tri thức và hành động nhắc việc. Tri thức và đồ thị đặt ghi chú có URL cạnh bản đồ khái niệm. Thực nghiệm có biểu đồ tương tác để đổi giữa AUC, Average Precision và F1, cùng bảng/đường cong chi tiết. Hướng dẫn cung cấp các lệnh khởi chạy và tái lập.

Mặc định giao diện dùng hồ sơ mẫu lấy từ train. Người dùng chọn Nhập JSON để thay 21 chỉ báo, chọn tác vụ và bấm Bắt đầu phân tích. Mục Nhật ký và số đo chứa gap, fatigue và glucose bổ sung. Các thẻ tác tử chuyển màu theo thông điệp thực tế của phiên vừa chạy; chi tiết được thu gọn trong mục Chi tiết phối hợp tác tử.



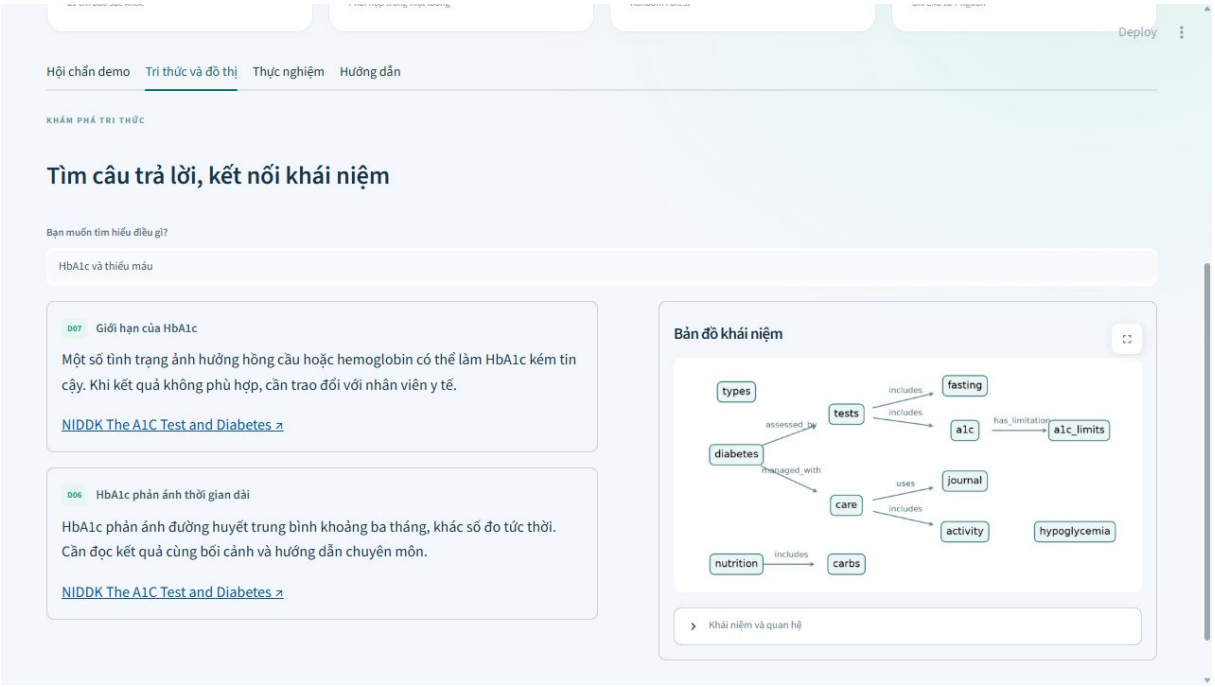
Hình 7.1: Giao diện local sau khi chạy hội chẩn tổng hợp. Ảnh chụp từ trình duyệt của bản Project01.

7.3 Ví dụ một lượt hội chẩn

Với câu hỏi về HbA1c và nhật ký, hồ sơ train minh họa, gap bằng 2 và fatigue bằng 0, hệ thống nhận diện không có điều kiện dừng ở Intake. Risk chạy model mặc định Random Forest, Knowledge truy xuất các ghi chú liên quan và Reminder đọc Q-table seed 42. Council kiểm tra các proposal rồi Advisor ghép kết quả.

Ở hồ sơ minh họa được chọn, score Random Forest khoảng 0,147 và ngưỡng 0,230 nên dự đoán nhãn 0. Giao diện trình bày score bằng vòng điểm, kèm model và ngưỡng. Tri thức được tách thành các thẻ có ID/URL; hành động Reminder nằm trong thẻ nhắc nhật ký. Có thể tải kết quả của cả phiên bằng nút Tải kết quả phiên.

Trace có 12 thông điệp để người đọc thấy thứ tự gọi, trạng thái từng bước và thời gian. Có thể tải JSON để kiểm tra proposal, evidence và lý do duyệt. Bảng trace không phải chuỗi suy nghĩ ẩn của model; nó là log của những thao tác phần mềm thực sự đã gọi.



Hình 7.2: Tab tri thức với query, ghi chú và URL nguồn để kiểm tra lại căn cứ.

7.4 Kịch bản bảo vệ ngắn

1. Giới thiệu ba nhánh học sâu, tri thức và RL; nêu rõ phạm vi giáo dục.
2. Chạy hội chẩn mẫu, chỉ score/ngưỡng, đoạn có ID nguồn, reminder và 12 thông điệp.
3. Chọn riêng MLP; mở notebook 01 để nối từ kiến trúc tới output train và model đã lưu.
4. Hỏi về HbA1c và thiếu máu, mở nguồn của ghi chú; giải thích truy xuất và graph một bước.
5. Nhập hồ sơ chỉ có BMI để cho thấy needs_input; sau đó thử số đo glucose 62 mg/dL để xem urgent trước Risk.
6. Mở kết quả RL, nêu trung thực Q-learning chưa vượt fixed. Dùng notebook 04 để trình diễn Council bác proposal thuốc hoặc ID giả.

Tài liệu `docs/KICH_BAN_DEMO.md` phân bổ khoảng tám phút cho các thao tác trên. Không dùng hồ sơ thật hoặc nhập thông tin riêng tư khi bảo vệ.

7.5 Tái lập thực nghiệm và notebook

```
python scripts/run_experiments.py
python scripts/execute_notebooks.py
python -m pytest -q --junitxml=artifacts/test-results.xml
python scripts/update_report_assets.py
```

Mã nguồn 7.2: Các lệnh tái lập độc lập với giao diện.

Các lệnh cần chạy bằng Python của môi trường đã cài dependencies. Script experiment có thể chạy riêng stage models, rl hoặc evaluation. Notebook 01 và 03 train thật nên khi Run All sẽ ghi lại artifact. Không nên mở quá trình train cùng lúc với demo vì app có thể đọc file trong lúc đang ghi. Sau train, khởi động lại app để nạp model mới.

Nếu muốn dùng Jupyter tương tác, chạy `python -m notebook`, mở theo thứ tự 01–04 và chọn đúng kernel. Mỗi notebook bắt đầu bằng cell in phiên bản Python và xác định thư mục project. Script build_notebooks chỉ dùng để bảo trì phần source notebook, không tạo output thực và sẽ xóa output cũ nếu chạy lại.

7.6 Xử lý lỗi thường gặp

Bảng 7.1: Lỗi vận hành và cách xử lý tương ứng.

Biểu hiện	Cách xử lý
Thiếu module	Cài requirements bằng đúng interpreter chạy app; tránh trộn env Anaconda và venv.
Lỗi nạp weight/joblib	Đối chiếu phiên bản đã ghi; dùng env tương thích hoặc huấn luyện lại từ CSV.
Thiếu manifest hoặc Q-table	Chạy script experiment trước khi khởi động UI.
CSV khác checksum	Khôi phục dữ liệu bàn giao; không sửa checksum chỉ để bỏ kiểm tra.
Notebook không tìm thấy module	Mở Jupyter từ Project01 hoặc notebooks trong Project01, kiểm tra cell setup.
Score không thay sau train	Dừng và mở lại server để xóa cache resource.
Sai mã Age	Dùng nhóm 1–13 theo README dữ liệu; không nhập số tuổi theo năm.

7.7 Biên dịch lại báo cáo

Source LaTeX nằm trong `report/`. Sau khi đổi artifact, chạy script update_report_assets để cập nhật bảng và hình đo được. Đọc lại phần diễn giải vì script không tự sửa kết luận trong văn bản. Dùng XeLaTeX hai lượt để cập nhật mục lục, tham chiếu và số trang. README trong thư mục report mô tả các lệnh cụ thể.

Phần 8

Kết luận và hướng phát triển

8.1 Kết quả đạt được

Project01 đã hiện thực một Diabetes Agent chạy local với mã nguồn độc lập, bốn notebook có output thực, dữ liệu có provenance, model đã huấn luyện và giao diện quan sát được quá trình phối hợp.

Kết quả supervised cho thấy các mô hình học được có khả năng phân biệt nhãn trên test nội bộ cao hơn Dummy. Random Forest được chọn mặc định theo AP validation; MLP vẫn được cung cấp và đánh giá đầy đủ. Truy xuất tìm đúng các ID đích trong bộ câu tự xây, còn bộ hợp đồng và pytest xác nhận những nhánh chức năng đã định. Q-learning học được policy có reward hơn never/rule nhưng chưa vượt fixed trong simulator hiện tại.

Giá trị của thiết kế đa tác tử ở project là trách nhiệm được tách rõ, có thông điệp, kiểm tra và quyền từ chối trước khi tổng hợp. Chưa có kết quả chứng minh kiến trúc này tốt hơn một pipeline đơn dùng cùng công cụ. Việc trình bày giới hạn là cần thiết để tách khả năng chạy được của phần mềm khỏi độ tin cậy trong ứng dụng thực.

8.2 Giới hạn hiện tại

Dữ liệu là khảo sát Hoa Kỳ, nhãn gộp và chưa có thẩm định ngoài mẫu. Mã nhóm nhân khẩu được đưa vào dưới dạng số; chưa kiểm tra thiên lệch theo nhóm hoặc phương án mã hóa khác. Supervised mới có một split/seed và chưa có khoảng tin cậy hoặc external validation.

KB nhỏ, bộ câu hỏi tự xây; Council và luật tiếp nhận chỉ kiểm tra các điều kiện đã định. RL dùng reward tự đặt và chưa có dữ liệu hành vi thật. Demo chưa có các yêu cầu vận hành cho hồ sơ bệnh nhân và không thay thế kế hoạch chăm sóc.

8.3 Hướng phát triển có thể kiểm chứng

Bước tiếp theo nên ưu tiên một bộ câu hỏi độc lập và đánh giá thủ công độ liên quan của trích đoạn, trước khi tăng số agent hoặc bổ sung LLM. Với nhánh dự đoán, cần so sánh mã hóa biến, lặp split/seed, báo khoảng bất định, đánh giá calibration và subgroup trong điều kiện sử dụng xác định.

Với RL, cần kiểm tra độ nhạy theo hệ số reward và động lực phản ứng, đối chiếu với nghiệm tối ưu của MDP nhỏ nếu muốn phân tích sai lệch policy. Chỉ chuyển sang nghiên cứu nhắc việc thực tế khi có dữ liệu, đồng thuận và thiết kế đánh giá phù hợp. Những mở rộng này tạo câu hỏi kiểm chứng được, thay vì chỉ tăng độ phức tạp của hệ thống.

Tài liệu tham khảo

[1] M. Wooldridge và N. R. Jennings, *Intelligent agents Theory and practice*, The Knowledge Engineering Review, 10(2), 1995. https://www.cs.ox.ac.uk/people/michael.wooldridge/pubs/ker95.pdf.

[2] C. J. C. H. Watkins và P. Dayan, *Q-learning*, Machine Learning, 8, 279–292, 1992. https://www.gatsby.ucl.ac.uk/~dayan/papers/cjch.pdf.

[3] UCI Machine Learning Repository, *CDC Diabetes Health Indicators*, dataset 891, metadata và CSV. https://archive.ics.uci.edu/dataset/891/cdc+diabetes+health+indicators. Kiểm tra 05/11/2026.

[4] PyTorch, *BCEWithLogitsLoss documentation*. https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html. Kiểm tra 05/11/2026.

[5] scikit-learn, *average_precision_score documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html. Kiểm tra 05/11/2026.

[6] NIDDK, *What Is Diabetes*. https://www.niddk.nih.gov/health-information/diabetes/overview/what-is-diabetes. Kiểm tra 05/11/2026.

[7] NIDDK, *Diabetes Tests and Diagnosis*. https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis. Kiểm tra 05/11/2026.

[8] NIDDK, *The A1C Test and Diabetes*. https://www.niddk.nih.gov/health-information/diagnostic-tests/a1c-test. Kiểm tra 05/11/2026.

[9] NIDDK, *Managing Diabetes*. https://www.niddk.nih.gov/health-information/diabetes/overview/managing-diabetes. Kiểm tra 05/11/2026.

[10] CDC, *Treatment of Low Blood Sugar Hypoglycemia*. https://www.cdc.gov/diabetes/treatment/treatment-low-blood-sugar-hypoglycemia.html. Kiểm tra 05/11/2026.

[11] CDC, *Diabetes Meal Planning*. https://www.cdc.gov/diabetes/healthy-eating/diabetes-meal-planning.html. Kiểm tra 05/11/2026.

[12] CDC, *Get Active*. https://www.cdc.gov/diabetes/living-with/physical-activity.html. Kiểm tra 05/11/2026.

[13] Trần Văn Khánh, *Báo cáo Project01 Hệ đa agent hỗ trợ sàng lọc và tư vấn bệnh tiểu đường*, tài liệu người dùng cung cấp, 36 trang, tháng 11/2026. Tham khảo cấu trúc bài toán; không dùng số liệu của tài liệu làm kết quả Project01 này.

[14] Bùi Anh Đức, *Báo cáo Agent01 và Agent02*, tài liệu có sẵn trong workspace INTEL_SYS, tháng 10/2026. Tham khảo bố cục và phong cách trình bày.